TractionTherm

Django backend design · modules, models, tables, services and APIs

This document describes the server that will replace the browser-only storage of the TractionTherm motor-monitoring web UI. It is written for the backend team: it lists the Django apps to create, every model and database table with its fields, the services that hold the business rules, and the REST and WebSocket APIs the existing front end needs.

Everything here is derived from what the current front end does today (the pages in this folder and `assets/js/app.js`), so the backend can be built without changing how the app looks or behaves.

Item	Value
Product	TractionTherm · traction-motor temperature monitoring (heat-run tests)
Front end	Static HTML / CSS / JavaScript, 12 pages, no framework
Target backend	Python 3.12, Django 5.x, Django REST Framework, PostgreSQL 16
Real-time	Django Channels (WebSockets) with Redis
Background work	Celery with Redis (evaluation, archiving, exports, device timeouts)
Status	Design for implementation · version 1.0

Contents

1. What the system does 4

1.1 Domain in one page . . . 4

1.2 Pages and what each needs from the server . . . 4

2. Architecture 5

2.1 Technology choices . . . 5

2.2 Project layout . . . 5

3. Modules (Django apps) 6

4. Models and database tables 6

4.1 accounts . . . 6

4.2 devices . . . 7

4.3 motors . . . 8

4.4 testing . . . 8

4.5 telemetry . . . 9

4.6 conditions . . . 9

4.7 activity . . . 10

4.8 Relationships at a glance . . . 10

5. Services (business logic) 11

5.1 Evaluation rules (port of the front-end logic) . . . 11

6. REST API 12

6.1 Auth and current user . . . 12

6.2 Devices . . . 12

6.3 Motors . . . 13

6.4 Tests (runs) . . . 13

6.5 Results and exports . . . 14

6.6 Settings . . . 14

6.7 Users, roles, activity . . . 14

6.8 Device ingestion (machine to machine) . . . 14

7. Live updates (WebSockets) 15

8. Permissions 15

9. Moving the front end to the API 16

10. Security and operations 16

11. Build plan 17

Acceptance tests to port 17

1. What the system does

Traction motors are heat-run tested in a loco shed. A test device sits at the bench and reads temperatures; the web app shows them live, judges each motor's condition and keeps the results.

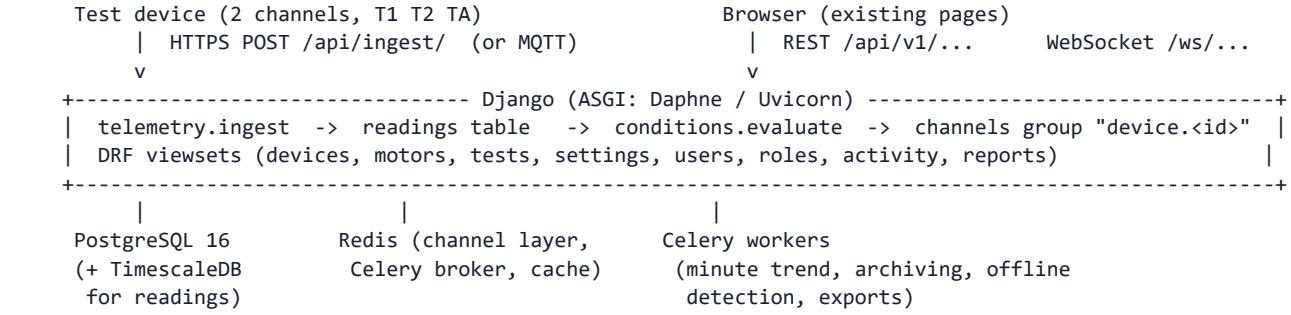
1.1 Domain in one page

- **Device**: a Wi-Fi data logger with **2 channels**. It can test at most 2 motors at once and has **5 terminals**: 2 per channel (T1, T2) and 1 shared ambient sensor (TA).
- **Motor**: identified by Motor ID and a unique **Stator No.** (plus model and locomotive). A motor can be tested many times over its life.
- **Test** (a run): one motor on one device channel, from Switch on until Stop. Every test has its own **Test ID** (e.g. TEST-0007). The stator stays the same; the Test ID changes each run.
- **Reading**: one sample of T1, T2 and TA every 2 s. A per-minute trend is kept for the whole test.
- **Condition**: every reading is judged Good / Monitor / Not OK against limits set in Settings; a motor takes its **worst** check.
- **Switch on without a motor**: an operator can start reading a free channel straight away; the run gets a temporary ID (NEW-001) and the real motor is attached later.

1.2 Pages and what each needs from the server

Page	Purpose	Server data used
Sign in (index)	Username / e-mail and password	auth
Dashboard	Counts by condition, alarms, rise chart	live summary, conditions
Monitor	A live widget per device, recent motor activity per device	devices, live readings, activity
Device	3D view of one device, two channels side by side, test history, activity	device, runs, readings, aggregation, activity
Motor	One motor: live 3D, readings, real-time chart, aggregation, comparison, test history	motor, run, readings, evaluation
Results	Test results table (one row per test), readings log, rules, CSV / print, test graphs	tests, readings, exports
Devices	Register and edit devices, see their motors	devices CRUD
Motors	Register, edit, assign, delete motors	motors CRUD, assignment
Settings	Motor conditions (Good / Monitor / Not OK), terminal limits, sensor names	settings
Users	Accounts, enable / disable	users CRUD
Roles & permissions	Roles and their CRUD permissions	roles CRUD
Activity log	Who did what, filter by device / user / category, CSV	activity

2. Architecture



2.1 Technology choices

Concern	Choice	Why
Web framework	Django 5 + Django REST Framework	Admin, ORM, migrations, auth and permissions out of the box
Database	PostgreSQL 16; TimescaleDB extension for readings	A 2 s sample per channel is about 43,000 rows a day per device; hypertables keep it fast
Live updates	Django Channels + channels-redis	The UI refreshes every 2 s; push instead of polling
Background jobs	Celery + Redis, celery-beat	Minute trend, device-offline detection, large exports
Auth	Django session auth (same-site) or SimpleJWT for devices	Browser: secure HttpOnly session cookie. Devices: per-device API keys
API docs	drf-spectacular (OpenAPI 3)	Generated schema for the front end and device firmware
Exports	csv module; WeasyPrint for PDF	Results CSV / print match today's output

2.2 Project layout

```
tractiontherm/                # Django project
  settings/  base.py dev.py prod.py
  asgi.py  urls.py  celery.py
apps/
  accounts/      users, roles, permissions, sign-in
  devices/       devices, channels, device API keys
  motors/        motors (identity: Motor ID, Stator No.)
  testing/       test runs, Test ID sequence, switch on / stop / assign / channel / terminals
  telemetry/     readings, minute trend, ingestion, live push
  conditions/    limits, terminal limits, sensor names, evaluation, heating model
  reports/       results queries, aggregation, CSV and PDF exports
  activity/      audit log
  core/          shared mixins, permissions, pagination, errors
```

3. Modules (Django apps)

App	Owns	Depends on
accounts	User, Role, RolePermission, sign-in / sign-out, permission checks	core
devices	Device, DeviceApiKey, online status, device switcher list	accounts
motors	Motor (identity and details)	accounts
testing	TestRun, TestIdSequence; Switch on, Stop, Assign, Change channel, Change terminals, Edit test	devices, motors, telemetry, conditions, activity
telemetry	Reading (2 s), TrendPoint (1 min), ingestion endpoint, live WebSocket push	devices, testing
conditions	ConditionLimits, TerminalLimit, SensorNames; evaluation and heating model	telemetry
reports	Results queries, aggregation buckets, CSV / PDF export	testing, telemetry, conditions
activity	ActivityEntry (append only) and its device tags	accounts
core	BaseModel (timestamps), permission classes, error format, pagination	-

4. Models and database tables

Field types are Django field types. Table names use the `app_model` default. Every table also has `created_at` and `updated_at` from `core.BaseModel` unless stated.

4.1 accounts

User

Table `accounts_user`

Field	Type	Rules / notes
id	BigAutoField	Primary key
username	CharField(150)	Unique, lower case; sign-in accepts username or e-mail
email	EmailField	Unique
name	CharField(120)	Display name, e.g. Arun Prakash
role	ForeignKey(Role, PROTECT)	Exactly one role per user
password	CharField	Django hash (PBKDF2 / Argon2). Never stored or sent in plain text
is_active	BooleanField	Disabled accounts cannot sign in; open sessions are ended
last_login	DateTimeField(null)	Set at sign-in

- Custom user model (`AUTH_USER_MODEL`), set before the first migration.

Role

Table `accounts_role`

Field	Type	Rules / notes
id	BigAutoField	Primary key
name	CharField(60)	Unique. Built in: Administrator, Supervisor, Operator, Viewer
description	CharField(200)	

Field	Type	Rules / notes
is_builtin	BooleanField	Built-in roles cannot be deleted or renamed
is_admin	BooleanField	Administrator: always every permission, cannot be edited

- Custom roles are created by an administrator; a role in use by a user cannot be deleted.

RolePermission

Table `accounts_rolepermission`

Field	Type	Rules / notes
role	ForeignKey(Role, CASCADE)	
key	CharField(40)	One of the permission keys in section 8, e.g. motors.update

- Unique together (role, key). Granting create / update / delete also grants the area's view (the UI already enforces this).

4.2 devices

Device

Table `devices_device`

Field	Type	Rules / notes
id	CharField(20), primary key	Device ID, e.g. TMU-01 (printed on the unit)
name	CharField(60)	Unique, e.g. Device A
location	CharField(120)	e.g. Loco Shed · Bay 3
ip_address	GenericIPAddressField(null)	Last reported
firmware	CharField(20)	e.g. 2.4.1
channels	PositiveSmallIntegerField	Default 2 (CH 1, CH 2)
signal	PositiveSmallIntegerField(null)	Wi-Fi bars 0-4, last reported
last_seen	DateTimeField(null)	Updated by every ingest; online = seen within 10 s

- Online / offline is derived from last_seen (a Celery beat task marks devices offline and pushes the change).

DeviceApiKey

Table `devices_deviceapikey`

Field	Type	Rules / notes
device	ForeignKey(Device, CASCADE)	
key_hash	CharField(128)	Hash of the key the device sends in its Authorization header
label	CharField(60)	
revoked_at	DateTimeField(null)	

4.3 motors

Motor

Table `motors_motor`

Field	Type	Rules / notes
id	BigAutoField	Primary key (internal)
motor_id	CharField(40)	Unique, upper case, e.g. TM-4471. Can be renamed; history follows the row
stator_no	CharField(40, null)	Unique when set, upper case, e.g. ST-60681. Required except for a temporary motor
model	CharField(120)	e.g. 6FRA 6068 · 3-phase AC
locomotive	CharField(60)	e.g. WAG-9 #31245
is_temporary	BooleanField	True for NEW-001 style motors created by Switch on before details are known

- Where a motor is now is not stored here: it is the device of its open TestRun, if any.
- Deleting a motor is refused while it has an open test; finished tests keep the motor (soft delete with `deleted_at` recommended).

4.4 testing

TestRun

Table `testing_testrun`

Field	Type	Rules / notes
id	BigAutoField	Primary key
test_id	CharField(24)	Unique, upper case, 3-24 letters / digits / dashes, e.g. TEST-0007
motor	ForeignKey(Motor, PROTECT)	
device	ForeignKey(Device, PROTECT)	Device the test ran on (editable later)
channel	PositiveSmallIntegerField(null)	1 or 2
state	CharField(10)	running / stopped / archived
started_at	DateTimeField	
stopped_at	DateTimeField(null)	Set by Stop
t1_measures	CharField(40)	What T1 measures in this test (default from Settings), e.g. Drive end (DE)
t2_measures	CharField(40)	What T2 measures in this test
result	CharField(8, null)	ok / warn / crit at the end of the test
max_t1, max_t2, max_rise	DecimalField(5,1)	Peaks, kept for the summary and fast lists
started_by, stopped_by	ForeignKey(User, SET_NULL, null)	

- **One running test per channel**: partial unique index on (device, channel) WHERE state = 'running'.
- **One open test per motor**: partial unique index on (motor) WHERE state IN ('running', 'stopped').
- A stopped test is the motor's 'current' test until a new test starts; then it becomes archived.

- A Test ID once used (current or archived) can never be used again.

TestIdSequence

Table `testing_testidsequence`

Field	Type	Rules / notes
id	SmallAutoField	Single row
last_number	PositiveIntegerField	Highest TEST-NNNN number used; next free is last_number + 1, zero padded to 4

- Read and increment inside `select_for_update()`. Custom Test IDs typed by users don't move the sequence unless they follow TEST-NNNN.

4.5 telemetry

Reading

Table `telemetry_reading`

Field	Type	Rules / notes
test_run	ForeignKey(TestRun, CASCADE)	Indexed together with time
time	DateTimeField	Device time, UTC; TimescaleDB partition column
t1, t2, ta	DecimalField(5,1)	°C, one decimal (PT100-class sensors)

- Written every 2 s per running channel; index (test_run, time). Raw 2 s data kept 30 days (retention policy).

TrendPoint

Table `telemetry_trendpoint`

Field	Type	Rules / notes
test_run	ForeignKey(TestRun, CASCADE)	
minute	DateTimeField	Start of the minute; unique together (test_run, minute)
t1, t2, ta	DecimalField(5,1)	Reading at that minute (what the charts, aggregation and downloads use)

- Kept for the life of the test (the front end keeps at most 600 points per run; the server has no such limit).

4.6 conditions

ConditionLimits

Table `conditions_conditionlimits`

Field	Type	Rules / notes
temp_monitor / temp_notok	DecimalField(5,1)	Motor temperature (hotter of T1 / T2) °C. Default 80 / 100
rise_monitor / rise_notok	DecimalField(5,1)	Rise above ambient K. Default 25 / 30
diff_monitor / diff_notok	DecimalField(5,1)	T1 / T2 difference °C. Default 10 / 15
partner_monitor / partner_notok	DecimalField(5,1)	Hotter than the other motor on the device, K. Default 8 / 12

Field	Type	Rules / notes
updated_by	ForeignKey(User, SET_NULL, null)	

- Single row. Every pair: both > 0 and Monitor < Not OK. Good = below Monitor.

TerminalLimit

Table `conditions_terminallimit`

Field	Type	Rules / notes
slot	CharField(2), primary key	T1, T2 or TA
min_c, max_c	DecimalField(5,1)	-40 to 250, min < max. Default T1 / T2 0-120, TA 0-50. Outside = Not OK

SensorNames

Table `conditions_sensornames`

Field	Type	Rules / notes
t1_name, t2_name	CharField(12)	Default T1, T2. 1-12 letters, digits, space . _ # -; not TA; must differ
t1_measures, t2_measures	CharField(40)	Default 'Drive end (DE)', 'Non-drive end (NDE)'. New tests copy these

- Single row. Names are used in every table header and download, so they are global.

4.7 activity

ActivityEntry

Table `activity_activityentry`

Field	Type	Rules / notes
id	BigAutoField	
time	DateTimeField	Server time, indexed
category	CharField(10)	auth, device, motor, test, limits, user, role, report, access
text	TextField	Human sentence, e.g. 'Stopped monitoring TM-4471 on Device A CH 1 · test TEST-0001'
user	ForeignKey(User, SET_NULL, null)	Plus copies of user name, username and role at the time
devices	ManyToManyField(Device)	Devices the event belongs to (both devices for a move). Powers the per-device lists
motor, test_run	ForeignKey(null, SET_NULL)	Optional links for 'open the motor' and filtering

- Append only: no update or delete through the API or admin. No updated_at.

4.8 Relationships at a glance

```
Role 1 --- * User                Device 1 --- * TestRun * --- 1 Motor
Role 1 --- * RolePermission      TestRun 1 --- * Reading
                                 TestRun 1 --- * TrendPoint
ActivityEntry * --- * Device     ActivityEntry * --- 0..1 Motor / TestRun / User
ConditionLimits (1 row)  TerminalLimit (3 rows)  SensorNames (1 row)  TestIdSequence (1 row)
```

5. Services (business logic)

Views stay thin: they validate input with serializers and call a service. Every service that changes data runs in `transaction.atomic()`, checks the permission, writes an ActivityEntry and publishes a WebSocket event.

Service	Function	Rules it enforces
TestIdService	next_id(), reserve(test_id)	Format and uniqueness against all current and archived tests; sequence under row lock
MonitoringService	switch_on(device)	Device online; a free channel; creates a temporary motor NEW-NNN and a running test with the next Test ID
	start(motor, device, test_id)	Free channel on that device; archives the motor's previous stopped test; copies sensor 'measures' defaults into the test
	stop(test_run)	Sets stopped_at, computes result and peaks; channel becomes free
AssignmentServic e	assign(motor, device, test_id)	If the motor runs elsewhere: stop there first (UI asks to confirm); then start on the target
	claim(temp_test, motor, test_id)	'Add details' on a switched-on run: the chosen motor takes over the run and readings; the temporary motor is removed
ChannelService	change(test_run, channel)	Same device. Other channel free: move. Busy: swap the two running tests. Readings stay with each test
TerminalService	change(test_run, t1_measures, t2_measures, swap)	swap=true exchanges t1 / t2 in all Reading and TrendPoint rows of that test and its peaks
TestEditService	edit(test_run, ...)	Edit test window: Test ID, device, channel, terminals, swap, and motor details (which apply to all the motor's tests). Moving a running test to another device needs a free channel there and the device online
IngestService	ingest(device, samples)	Authenticates the device key; maps channel to its running test; stores readings; updates last_seen; rolls the minute trend; triggers evaluation and push
EvaluationService	evaluate(test_run)	Checks and condition (section 5.1); heating model; returns the same structure the UI shows
ReportService	results(filters), readings(filters), export_csv(), export_pdf()	One row per test; readings log sampled at an interval; rules (all / any)
ActivityService	log(category, text, user, devices, motor, test)	Tags devices from the event; immutable
AuthService	sign_in(), sign_out()	Username or e-mail; disabled accounts refused; failed attempts logged and rate limited

5.1 Evaluation rules (port of the front-end logic)

Check	Value	Monitor / Not OK
Motor temperature	hot = max(T1, T2)	temp_monitor / temp_notok
Rise above ambient	rise = hot − TA	rise_monitor / rise_notok
T1 / T2 difference	|T1 − T2|	diff_monitor / diff_notok
Predicted final rise	Heating-model final rise A (when confident)	Monitor if A ≥ rise_notok
Compared with the other motor	rise − partner rise (only when both ran ≥ 15 min and started within 15 min of each other)	partner_monitor / partner_notok
Terminal limits	each of T1, T2, TA within its min / max	Any outside = Not OK
Probe readings plausible	T1 or T2 below TA − 3 °C or above 200 °C	Monitor (check probe)

A motor's condition is the worst of its checks. A stopped test keeps its last condition as its **result**; a device offline shows 'Offline'.

Heating model

For each of T1 and T2 the rise above ambient over test time t (minutes) is fitted to **rise(t) = A − (A − r0)·e$^{-t/\tau}$** by least squares (grid search on τ from 1 to 600 min, closed form for A and r0). The fit is **confident** when $R^2 \geq 0.8$ and the test has run at least 0.4·τ. From it: predicted final temperature = TA + A; time to equilibrium t = τ·ln(60·(A − r0) / (2·τ)); time to the alarm rise. **Thermal equilibrium (IEC 60034-1)** is measured: the rise changes by less than 2 K per hour over two successive 15-minute windows.

> Run evaluation in a Celery task after each minute trend point and cache the latest result per test in Redis; the live push every 2 s re-uses the cached model and only recomputes the simple checks.

6. REST API

Base path `/api/v1/`. JSON, UTC ISO-8601 times, temperatures in °C with one decimal. Errors use `{"error": {"code": "...", "message": "...", "field": "..."}}` with 400 (validation), 401 (not signed in), 403 (no permission), 404, 409 (conflict such as a busy channel or a used Test ID).

6.1 Auth and current user

Method	Path	Permission	Does
POST	/auth/sign-in/	public	Body: login (username or e-mail), password. Sets the session cookie; returns the user with role and permission keys
POST	/auth/sign-out/	signed in	Ends the session
GET	/auth/me/	signed in	Current user, role, permission keys (the UI hides buttons with these)

6.2 Devices

Method	Path	Permission	Does
GET	/devices/	devices.view or monitor.view	List with online state, channels in use, running motors
POST	/devices/	devices.create	Create (id, name, location, ip, firmware)
GET	/devices/{id}/	devices.view	One device with both channels and its stopped motors
PATCH	/devices/{id}/	devices.update	Edit
DELETE	/devices/{id}/	devices.delete	Refused while a test is running on it
POST	/devices/{id}/switch-on/	monitoring.create	Start reading the free channel without a motor (temporary NEW-NNN)
GET	/devices/{id}/activity/?limit=5	activity.view	Recent motor and test events of this device

6.3 Motors

Method	Path	Permission	Does
GET	/motors/?device=&search=&condition=	motors.view	List with where it is, its current test and live condition
POST	/motors/	motors.create	Register (motor_id, stator_no, model, locomotive); optional device + test_id to start straight away
GET	/motors/{id}/	monitor.view	Details, current test, live readings and evaluation
PATCH	/motors/{id}/	motors.update	Edit details (applies to all its tests)
DELETE	/motors/{id}/	motors.delete	Refused while it has an open test
POST	/motors/{id}/assign/	motors.update	Body: device, test_id. Starts a test on the device's free channel (moves it if running elsewhere)
GET	/motors/{id}/tests/	monitor.view	Its tests, newest first (Test history)

6.4 Tests (runs)

Method	Path	Permission	Does
GET	/tests/?motor=&device=&condition=&search=	reports.view	Results table: one row per test (current and earlier)
GET	/tests/{test_id}/	reports.view	One test with summary, peaks, terminals, result
PATCH	/tests/{test_id}/	motors.update	Edit test: test_id, device, channel, t1/t2 measures, swap, motor details
POST	/tests/{test_id}/start/	monitoring.update	Switch a stopped motor on again (creates a new test with the next Test ID)
POST	/tests/{test_id}/stop/	monitoring.update	Stop
POST	/tests/{test_id}/claim/	motors.update	Add details on a switched-on run: body motor (existing) or new motor fields, test_id
POST	/tests/{test_id}/channel/	monitoring.update	Body: channel. Moves or swaps
POST	/tests/{test_id}/terminals/	monitoring.update	Body: t1_measures, t2_measures, swap
GET	/tests/{test_id}/readings/?from=&to=&every=	reports.view	Trend points (or raw 2 s samples for the last 5 minutes with every=2s)
GET	/tests/{test_id}/aggregation/?step=10	reports.view	Min / avg / max per 5, 10 or 15-minute interval for T1, T2, TA
GET	/tests/{test_id}/evaluation/	monitor.view	Checks, condition, heating model
GET	/tests/{test_id}/export.csv	reports.create	This test's full results
GET	/test-ids/next/	monitoring.create	Next free Test ID to pre-fill forms

6.5 Results and exports

Method	Path	Permission	Does
GET	/reports/readings/?range=60&every=5&...	reports.view	Readings log (Temperature readings view)
POST	/reports/export/	reports.create	Body: type (summary / readings), filters, rules; returns CSV (or PDF for print)
GET	/motors/{id}/export.csv	reports.create	Every test of one motor in one file
GET	/reports/all-motors.csv	reports.create	All motor results

6.6 Settings

Method	Path	Permission	Does
GET	/settings/	limits.view	Condition limits, terminal limits, sensor names
PUT	/settings/conditions/	limits.update	Monitor / Not OK pairs (validated)
PUT	/settings/terminals/	limits.update	Min / max for T1, T2, TA
PUT	/settings/sensors/	limits.update	Sensor names and default 'measures'

6.7 Users, roles, activity

Method	Path	Permission	Does
GET / POST	/users/	users.view / users.create	List, create
PATCH / DELETE	/users/{id}/	users.update / users.delete	Edit, enable / disable, delete (not yourself, not the last administrator)
GET / POST	/roles/	roles.view / roles.create	List with permission keys, create custom role
PATCH / DELETE	/roles/{id}/	roles.update / roles.delete	Built-in roles can't be deleted; Administrator can't be edited
GET	/permissions/	roles.view	The permission catalogue (area, action, label)
GET	/activity/?category=&user=&device=&period=&search=	activity.view	Paged, newest first
GET	/activity/export.csv	activity.view	Same filters

6.8 Device ingestion (machine to machine)

Method	Path	Permission	Does
POST	/ingest/	device API key	Body below. 204 on success; 409 if the channel has no running test (samples are dropped and reported)
POST	/ingest/heartbeat/	device API key	ip, firmware, signal; keeps the device online when idle

```
POST /api/v1/ingest/                Authorization: Device <api-key>
{
  "device": "TMU-01",
  "samples": [
    {"time": "2026-11-02T06:31:04Z", "channel": 1, "t1": 50.2, "t2": 53.1, "ta": 31.0},
    {"time": "2026-11-02T06:31:04Z", "channel": 2, "t1": 57.0, "t2": 59.0, "ta": 31.0}
  ]
}
```

Devices may batch several seconds of samples. The same TA reading is sent with both channels (one ambient sensor per device).

7. Live updates (WebSockets)

Socket	Joined by	Messages pushed
/ws/live/	Monitor, Dashboard	device.status (online, signal), reading (device, channel, t1, t2, ta, condition) every 2 s, test.changed
/ws/devices/{id}/	Device page	readings for both channels, test.changed, activity.new
/ws/tests/{test_id}/	Motor page	reading, evaluation (every minute), test.changed
/ws/activity/	Monitor (per-device lists)	activity.new with its device ids

Sockets use the same session as the REST API and check `monitor.view` (and `activity.view` for activity). Groups: `device.<id>`, `test.<test_id>`, `activity`.

8. Permissions

Permissions are keys `area.action`. The server checks them on every request (a DRF permission class reads the key from the view); the front end only hides buttons. Administrator always has all keys.

Key	Allows	Admin	Super.	Oper.	Viewer
monitor.view	Live readings, dashboard, motor pages	Yes	Yes	Yes	Yes
monitoring.create	Switch on a free channel	Yes	Yes	Yes	
monitoring.update	Start / stop, change channel, change terminals	Yes	Yes	Yes	
motors.view / create / update / delete	Motors page, register, edit (incl. Edit test, assign), delete	Yes	Yes	view	view
devices.view / create / update / delete	Devices and device pages, add, edit, delete	Yes	Yes	view	view
limits.view / update	Settings page, change conditions, terminals, sensor names	Yes	Yes	view	view
reports.view / create	Results page, download and print	Yes	Yes	Yes	view
activity.view	Activity log and recent-activity lists	Yes	Yes		
users.view / create / update / delete	Users	Yes			
roles.view / create / update / delete	Roles & permissions	Yes			

Defaults match the current front end; administrators can change any role except Administrator and create custom roles.

9. Moving the front end to the API

Today every page keeps its data in the browser's localStorage. Each key maps to an endpoint; replace the `store.get / store.set` calls in `app.js` with fetch calls and the simulated `tick()` with the WebSocket.

Browser key today	Holds	Replace with
tt-devices-v6	Devices, their motors, current tests, test history	/devices/, /motors/, /tests/
tt-spare-v1	Motors not on a device	/motors/?unassigned=1
tt-test-seq	Last Test ID number	/test-ids/next/
tt-limits-v3	Motor conditions	/settings/conditions/
tt-terminals-v1	Terminal min / max	/settings/terminals/
tt-sensors-v1	Sensor names and measures	/settings/sensors/
tt-users-v2, tt-roles-v1, tt-grants-v3	Users (with plain passwords), roles, permissions	/users/, /roles/, /auth/me/
tt-activity-v1	Activity log (max 2000)	/activity/
tt-session (sessionStorage)	Signed-in user id	Server session cookie + /auth/me/
tick() every 2 s	Simulated readings	/ws/live/ and /ws/tests/{id}/

The front end can keep all its screens. Only its data layer changes; the evaluation, heating model and aggregation move to the server so every user, report and export sees the same numbers.

10. Security and operations

- **Passwords**: hashed by Django; reset by an administrator or by e-mail link. The demo accounts in the front end must not be carried over.
- **Sessions**: HttpOnly, Secure, SameSite=Lax cookie; CSRF on all unsafe requests; sign-out and disabling a user end their sessions.
- **Brute force**: rate-limit sign-in (e.g. django-axes, 5 failures per 15 min) and log failures to the activity log.
- **Devices**: one API key per device, stored hashed, sent over HTTPS only; revoke from the admin.
- **Audit**: ActivityEntry is append only; restrict the Django admin to read-only for it.
- **Time**: store UTC; the UI shows local time (Asia/Kolkata).
- **Retention**: raw 2 s readings 30 days; minute trend, tests and activity kept (archive yearly).
- **Backups**: nightly PostgreSQL dump plus point-in-time recovery; test a restore monthly.
- **Deployment**: Docker images for web (ASGI), Celery worker, Celery beat; Nginx serving the static pages and proxying /api and /ws.
- **Monitoring**: alert when a device stops sending while a test is running, and on ingestion errors.

11. Build plan

Phase	Scope	Done when
1. Foundation	Project, apps, custom user, roles and permissions, sign-in, /auth/me/, admin	Users sign in on the server; the UI hides buttons from /auth/me/
2. Registry	Devices, motors, settings endpoints and validation	Devices, Motors and Settings pages read and write the API
3. Tests	TestRun, Test ID sequence, switch on, start, stop, assign, claim, channel, terminals, Edit test	All test actions work from the UI with the same rules
4. Telemetry	Ingest API, readings, minute trend, offline detection, WebSockets	A real device streams and Monitor / Motor pages update live
5. Evaluation	Checks, heating model, results per test, aggregation	Server conditions match the current UI on recorded data
6. Results and audit	Results queries, CSV / PDF export, activity with device tags	Results and Activity pages run on the API
7. Hardening	Rate limits, backups, retention, deployment, load test (10 devices × 2 channels)	Ready for the shed

Acceptance tests to port

The front end already has browser tests for every flow (sign-in, switch on, add details, assign, stop / restart, change channel and terminals, Edit test, Test ID rules, results per test, per-device activity, permissions for all four roles, page connectivity). Re-run them against the API build; the expected behaviour should not change.